

# Proyecto de Investigación

Hugo Missael González Cruz

22 de julio de 2026

## Índice

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>1. Introducción al Análisis de Datos</b>                                      | <b>2</b>  |
| <b>2. Análisis Exploratorio de Datos</b>                                         | <b>5</b>  |
| <b>3. Reducción de Dimensión</b>                                                 | <b>5</b>  |
| 3.1. PCA . . . . .                                                               | 6         |
| 3.1.1. La proyección lineal de un dato . . . . .                                 | 7         |
| 3.1.2. Las matrices de centrado y la proyección de múltiples datos . . . . .     | 9         |
| 3.1.3. La matriz de Covarianza y Descomposición en Vectores Propios . . . . .    | 10        |
| 3.1.4. El rango de la matriz de covarianza y los componentes a retener . . . . . | 13        |
| <b>4. Kernel PCA</b>                                                             | <b>15</b> |
| 4.1. Espacios de Hilbert con Kernel Reproducible (RKHS) . . . . .                | 15        |
| 4.2. Kernel PCA . . . . .                                                        | 19        |
| 4.2.1. Preeliminares . . . . .                                                   | 19        |
| 4.2.2. Proyección en el espacio de características . . . . .                     | 21        |
| <b>5. Ejemplo</b>                                                                | <b>25</b> |
| 5.1. Preeliminares . . . . .                                                     | 25        |
| 5.1.1. Kernels usuales . . . . .                                                 | 25        |
| 5.1.2. Error de Reconstrucción y <i>Trustworthiness</i> . . . . .                | 26        |
| 5.2. Desenvolviendo una curva S . . . . .                                        | 26        |
| <b>6. Conclusiones</b>                                                           | <b>30</b> |

# 1. Introducción al Análisis de Datos

Recientemente ha habido una explosión en la cantidad de datos que se recopilan sobre diversos aspectos de nuestra vida. Las actividades que realizamos en línea como leer noticias, escuchar música o comprar, están siendo registradas constantemente y generan una cantidad inmensa de datos. Además, actualmente, contamos con gran poder computacional accesible y a bajo costo, en este contexto, ha surgido la necesidad de procesar los datos para convertirlos ya sea en productos o en decisiones accionables. Puede definirse la ciencia de datos como la intersección de matemáticas, estadística y programación, con el objetivo de descubrir patrones en los datos, esto —claro— sin ser únicamente una o la otra. Según Garzon y col. (2022):

Data science is about solving problems based on observations of factors (referred to as co-variates, predictors, or just features) that may determine a solution.

Tratándose de la materia prima de la disciplina, nuestro punto de partida son los datos. Empezando por distinguir los objetos que consideramos datos, estableceremos nuestro objeto de análisis. Según Garzon y col. (2022):

Data is an objective recording of one or several event(s) in the real world that is accessible at later times for perusal and analysis by at least one person.

Es decir, un dato es un registro de un fenómeno del mundo real que es susceptible a ser analizado.

*Ejemplo 1.* Son Datos:

- Los contenidos de una página web.
- Mediciones (precisos o no) de algún fenómeno físico.
- Estadísticas recolectadas por sitios web.

No son datos:

- Los recuerdos de una persona sobre un evento.
- La ocurrencia de un evento (si no se registra).
- Los contenidos de la memoria RAM de una computadora.

Podemos clasificar a los datos de acuerdo a los valores que puede tomar. Un dato es *cualitativo* si describe información que no puede ser medida, por ejemplo, grado de estudios, género o religión. Es ordinal si es susceptible a ser ordenado y nominal en otro caso. Por otro lado, un dato es *cuantitativo* si representa información que puede ser medida y representada naturalmente con números, por ejemplo, edad, estatura o salario. Es discreto si toma valores de un conjunto finito o infinito numerable y es continua si toma valores sobre un conjunto no numerable. Podemos resumir la clasificación anterior en la siguiente tabla:

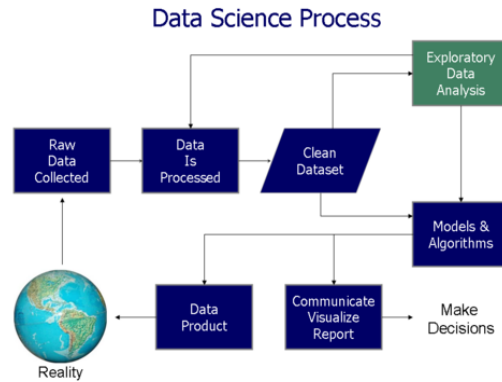


Figura 1: El proceso de análisis de datos. Farcaster at English Wikipedia, CC BY-SA 3.0 <https://creativecommons.org/licenses/by-sa/3.0>, via Wikimedia Commons.

1. Cualitativa
  - a) Ordinal
  - b) Nominal
2. Cuantitativa
  - a) Discreta
  - b) Continua

Decimos que los datos recolectados de un fenómeno del mundo real están en “crudo”. Los datos en dicho estado son susceptibles para ser analizados, pero tal vez no estén en una forma útil para nuestro análisis. A esta parte del proceso, se le conoce como limpieza de datos y consiste en transformarlos de modo que sean apropiados para su análisis.

A partir de aquí el proceso no es lineal y depende del objetivo para el que se hace el análisis. Una opción es hacer un análisis exploratorio de datos. En tal caso, podríamos hallar valores duplicados, faltantes o mal registrados; de forma que deberíamos regresar al paso anterior, es decir, limpiar de nuevo el conjunto de datos. Enseguida, dependiendo del problema que deseamos resolver, podemos usar algún modelo o algoritmo que nos ayude a resolver el problema. Finalmente, se interpretan los resultados y opcionalmente se comunican mediante reportes o visualizaciones. A partir de la cuales se toman decisiones, acciones o se desarrollan productos.

Una forma simple de organizar datos es en una tabla. En un arreglo de este tipo, las columnas se pueden considerar como vectores de dimensión  $n$  que describen **atributos** de los datos. A su vez, los renglones en la tabla son un vector de

dimensión  $p$  conformado por valores específicos de los atributos. Llamamos a una columna **atributo** (*feature*) y a un renglón, **registro** (*record*). Una tabla de este estilo o un conjunto de ellas se llama conjunto de datos (*dataset*).

De lo anterior, es claro que nuestro interés no son los datos aislados, sino un conjunto de ellos —extraídos de un fenómeno del mundo real— en el que podemos comenzar a buscar patrones. Las razones para analizar datos son muchas y muy variadas, pero —a grandes rasgos— se pueden clasificar en tres problemas: clasificación, predicción y *clustering*.

Finalmente, presentamos una clasificación típica de los tres principales tipos de problemas que podemos resolver con análisis de datos. Para ello, llamamos  $\Omega$  a la población concerniente al problema y  $D \subset \Omega$  una muestra relativamente pequeña (respecto a la población). Entonces, resolver el problema, consiste en obtener un modelo  $M$  en términos de  $D$  que soluciona el problema en los términos esperados.

*Ejemplo 2.* Considera el problema de clasificar flores Iris en tres categorías: Setosa, Versicolor y Virginica. En este caso,  $\Omega$  es el conjunto de flores Iris,  $D$  es el conjunto de flores Iris de las que hemos registrado datos como la longitud y anchura de sus pétalos. Entonces, una solución sería un algoritmo que, a partir de la longitud y anchura de los pétalos, sea capaz de clasificar una flor Iris en la especie correcta.

El anterior es un ejemplo de un problema de **clasificación**. Un problema de este tipo consiste en definir categorías sobre la población y, dada una entrada, decidir a cuál de ellas pertenece. Esto es, si  $\Pi$  es una partición de  $\Omega$ , dada una entrada  $x$ , encontrar  $C \in \Pi$  tal que  $x \in C$ .

Una solución a un problema de clasificación es un modelo que coloca a  $x$  en la categoría correcta.

Un problema de **predicción** consiste en, dada una función  $f : \Omega \rightarrow \mathbb{R}$  que le asigna un valor numérico a alguna característica de  $x \in \Omega$  y cuyo valor es difícil de medir directamente. El problema es hallar el valor de  $f(x)$ . Una solución es un modelo, basado en otras características de  $x$ , capaz de predecir el valor de  $f(x)$ .

Finalmente, un problema de *clustering* o de conglomerados, consiste en construir una partición  $\Pi$  en  $\Omega$ , dada una métrica  $d$  que mide el grado de similitud entre elementos de la población. Una solución es un modelo que produce una partición tal que elementos en un *cluster* son similares.

Con esto, tenemos una idea general del flujo de trabajo. Sin embargo, el enfoque de este trabajo son los algoritmos de reducción de dimensión que típicamente suelen considerarse concerniente a la fase de exploración de datos o a la fase de visualización.

## 2. Análisis Exploratorio de Datos

Usualmente, se presenta al análisis exploratorio de datos en contraparte al análisis confirmatorio de datos, este último, dedicado a las hipótesis y al modelado. Según Tukey (1977), se trata de observar los datos para ver qué es lo que parecen decir. Se considera que cualquier apariencia que observemos son descripciones parciales y trata de ver más allá. Está interesado en la apariencia y no en la confirmación.

Por lo anterior, el análisis exploratorio de datos no puede ser todo el análisis, es un primer paso. Una manera de encontrar pistas que guíen el análisis confirmatorio. Es un método sistemático para hacerse de una visión global del conjunto de datos. Para esto, utiliza gráficas, transforma las variables y presenta estadísticas que resumen el estado de los datos. Se trata de entender los datos, de hacerse una idea de su forma y ganar intuición. Sin embargo, no habría gran valor en explorar los datos si descubrimos lo que ya sabíamos. Un buen análisis exploratorio nos fuerza a ver lo que no esperábamos.

El conjunto de herramientas para hacer análisis exploratorio de datos incluye:

- Estadísticas que resumen el estado global del conjunto de datos.
- Histogramas, Diagramas de caja, diagramas de tallo y hoja.
- Visualizaciones que ayudan a entender la relación entre las variables.

Un problema que surge cuando se tienen una gran cantidad de datos es hacerla entendible. La visualización de datos es un enfoque del análisis exploratorio que utiliza métodos gráficos para hacer sentido del conjunto de datos. En especial, cuando el conjunto es grande o tienen una cantidad de variables. En dichos casos, las herramientas mencionadas anteriormente podrían no ser suficientes, entonces, se utilizan herramientas más sofisticadas como el *clustering* y la reducción de dimensión.

## 3. Reducción de Dimensión

En general, el objetivo de la Reducción de Dimensión es encontrar una representación en una dimensión más baja de los datos y, que a su vez, preserve las propiedades clave para un problema dado.

Podemos usar técnicas de reducción de dimensión para:

- Visualizar un conjunto de datos con muchas variables.
- Mitigar la *maldición de la dimensión*.
- Mitigar los efectos de la colinealidad.

La visualización puede llevarse a cabo al empezar —como parte del proceso de análisis exploratorio— o bien, como parte de la presentación de los resultados. En este contexto, la reducción de dimensión es útil para visualizar un conjunto

de datos con (posiblemente) decenas o cientos de variables. Ayudándonos a entender las relaciones entre las variables o a presentar visualmente los resultados de nuestro análisis.

Suponiendo que nuestras observaciones son mediciones de  $d$  características, es decir,  $x \in \mathbb{R}^d$ . Entonces, nuestro conjunto de datos es un conjunto de observaciones de dimensión  $d$ . Usualmente, no todas las características son igual de significativas, de modo que podríamos prescindir de ellas y conservar una cantidad significativa de la información original. En otras palabras, el conjunto de datos no cubre todo el espacio  $\mathbb{R}^d$ , pero yace en una estructura embebida de dimensión menor que  $d$ . En especial, la hipótesis de la variedad postula que los puntos de un conjunto de datos, yacen en una variedad (subvariedad o subespacio) de dimensión menor que  $d$ . Por lo anterior, a la reducción de dimensión también se le conoce como aprendizaje de variedades (subvariedades).

Desde este punto de vista, el aprendizaje de variedades se puede ver como una clase de algoritmos que intenta recuperar una variedad de dimensión baja inmersa en un espacio ambiente de dimensión alta.

Podemos clasificar las técnicas de reducción de dimensión en tres: métodos espectrales, métodos probabilísticos y métodos basados en redes neuronales. Los métodos espectrales intentan encontrar la variedad —ya sea lineal o no-lineal— de los datos. Usualmente, el problema se reduce a un problema de vectores propios generalizados. En los métodos probabilísticos, se considera que los datos son variables aleatorias multidimensionales. Es posible hallar una variable latente en la cual, la variable aleatoria, está condicionada. El problema es hallar una representación de menor dimensión de dicha variable latente. Finalmente, los métodos basados en redes neuronales utilizan un *autoencoder* donde los datos se comprimen entre el codificador y el decodificador, en ese punto los datos pasan por un cuello de botella de menor dimensión para luego ser reconstruidos, lo que hace al cuello de botella ideal para reducir la dimensión.

### 3.1. PCA

Análisis de Componentes Principales (PCA), es un método lineal de reducción de dimensión. Originalmente fue propuesto por Pearson en 1901 en el artículo “*On lines and planes of closest fit to systems of points in space*” (Pearson (1901)). Siguiendo con la hipótesis de la variedad, PCA asume que los datos en una dimensión grande  $x \in \mathbb{R}^D$  yacen en un subespacio lineal de  $\mathbb{R}^D$  e intenta encontrar el subespacio tal que la proyección y posterior reconstrucción de los datos tienen el menor error de reconstrucción.

Posteriormente, se desarrollaron otros métodos basados en PCA como dual PCA, PCA supervisado (SPCA) y kernel PCA. Este último generaliza una gran cantidad de métodos espectrales de reducción de dimensión mediante el uso de *kernels* y matrices de Mercer. Por ejemplo, usando el kernel  $K = X^\top X$  obtenemos PCA que se explica en la siguiente sección. Mientras que, con el kernel  $K = \frac{1}{2}HDH$  obtenemos MDS. Entre los métodos generalizados por kernel PCA

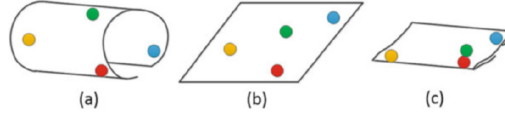


Figura 2: (a) La variedad original (b) La variedad desdoblada correctamente (c) El espacio lineal hallado por PCA

están: Principal Component Analysis (PCA), Multidimensional Scaling (MDS), Isomap, spectral clustering, Laplacian eigenmap, diffusion map, y Locally Linear Embedding (LLE).

Esto da pie a los métodos de *Maximum Variance Unfolding* (MVU) que buscan desdoblar la variedad a su máxima varianza sobre la dimensión intrínseca al conjunto de datos, es decir, buscan el mejor kernel para el conjunto de datos.

Lo anterior ejemplifica mejor el panorama de los métodos espectrales pero, a partir de ahora, limitamos la discusión a PCA.

Las ventajas son:

- Los componentes principales ( $\tilde{x}_i$ ) son combinaciones lineales de las variables originales ( $x_i$ ).
- Podemos proyectar nuevos datos en los ejes principales.
- Crea nuevas variables, no correlacionadas.

Mientras que algunas desventajas son:

- Funciona mejor si las variables están relacionadas linealmente.
- Normalmente, los componentes principales con valores propios más chicos son desechados, entonces, debemos decidir el número de componentes principales a retener.

Como se verá a continuación, PCA hace uso de la distancia euclidiana. Si los datos no yacen en una variedad lineal, es probable que PCA no “desdoble” apropiadamente la variedad, ver la figura (2).

En este caso, aún podemos usar PCA si aplicamos una transformación apropiada a los datos que los linearice. Aunque probablemente, sea más apropiado usar un método no lineal.

### 3.1.1. La proyección lineal de un dato

Consideramos un conjunto de datos  $\{(x_i, y_i) \mid i = 1, \dots, n\}$  con  $n$  el tamaño de la muestra y  $x_i \in \mathbb{R}^d$ ,  $y_i \in \mathbb{R}^l \forall i$ . El conjunto  $\{x_i\}_{i=1}^n$  contiene los datos de entrada, mientras que  $\{y_i\}_{i=1}^n$  contiene las observaciones.

Para futuras referencia, tenemos las siguientes definiciones

**Definición 1.** Definimos, el conjunto de entrenamiento como las siguientes matrices:

$$X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$$

$$Y = [y_1, \dots, y_n] \in \mathbb{R}^{l \times n}$$

De forma similar, si  $\{x_{t,i}\}_{i=1}^{n_t}$ , donde  $x_{t,i} \in \mathbb{R}^{n_t} \forall i$ , son datos fuera del conjunto de entrenamiento.

**Definición 2.** Definimos, los puntos fuera de la muestra de interés como

$$X_t := [x_{t,1}, \dots, x_{t,n_t}] \in \mathbb{R}^{d \times n_t}$$

Empecemos por proyectar un dato  $x \in \mathbb{R}^d$  en el espacio vectorial generado por los  $p < d$  vectores columna de la matriz  $U = [u_1, \dots, u_p]$  donde  $u_i \in \mathbb{R}^p$  y por lo tanto  $U \in \mathbb{R}^{d \times p}$ . Denotamos,  $col(U) \in \mathbb{R}^p$  al espacio generado por dichos vectores. Entonces, el problema de hallar la proyección  $\hat{x}$  en  $col(U)$ , consiste en determinar los coeficientes  $\beta \in \mathbb{R}^p$  en el sistema lineal:

$$\hat{x} = U\beta \tag{3.1}$$

Si  $x \in col(U)$ , entonces (3.1) tiene solución exacta,  $\hat{x} = x$ . En caso, contrario definimos los residuos como  $r = x - \hat{x} \neq 0$ , la diferencia entre el punto observado y la proyección (y posterior reconstrucción). Evidentemente, queremos que el vector de residuos sea mínimo. En tal caso, tenemos el siguiente teorema.

**Teorema 1.** *El residuo más pequeño es perpendicular  $col(U)$ .*

Con esto, podemos dar una expresión de los coeficientes en términos de  $U$ .

*Corolario 1.* Dado el vector mínimo de residuos, los coeficientes están dados por

$$\beta = (U^T U)^{-1} U^T x$$

*Demostración.* Sea  $r_{min}$  el residuo mínimo, entonces

$$r_{min} \perp U \implies x - U\beta \perp U$$

de donde

$$\begin{aligned} U^T(x - U\beta) &= 0 \\ U^T x - U^T U \beta &= 0 \\ U^T U \beta &= U^T x \end{aligned}$$

Por lo tanto,  $\beta = (U^T U)^{-1} U^T x$ . □



Reemplazando  $\beta$  en (3.1) obtenemos

$$\hat{x} = U(U^\top U)^{-1}U^\top x \quad (3.2)$$

A la matriz,  $\Pi := U(U^\top U)^{-1}U^\top$  se le conoce como matriz de proyección o matriz *hat*. Si además,  $\{u_1, \dots, u_p\}$  forman una base ortonormal, entonces  $U$  es ortogonal y por lo tanto  $U^\top U = I_p$ . De modo que  $\beta = U^\top x$  y (3.2) se reduce a

$$\hat{x} = UU^\top x \quad (3.3)$$

La proyección de  $x$  en  $\text{col}(U)$  se denota por  $\tilde{x}$ .

$$\tilde{x} = U^\top x \in \mathbb{R}^p \quad (3.4)$$

En PCA, es necesario que los datos estén centrados. En el caso de proyectar un solo dato, esto significa restar la media de las columnas. Dicha media está dada como sigue:

$$\mu_x = \frac{1}{n} \sum_{i=1}^n x_i \quad (3.5)$$

Así, denotamos al dato centrado como  $\check{x}$ , definido por

$$\check{x} = x - \mu_x \quad (3.6)$$

La ecuación (3.4) se convierte en

$$\tilde{x} = U^\top \check{x} \quad (3.7)$$

Finalmente, la reconstrucción en el subespacio (3.3) es, por lo tanto

$$\hat{x} = UU^\top x + \mu_x = \tilde{x} + \mu_x \quad (3.8)$$

Note que la media se agrega de nuevo en la reconstrucción, pues se restó en la proyección.

### 3.1.2. Las matrices de centrado y la proyección de múltiples datos

En PCA, es un requisito que todos los puntos estén centrados. Por esta razón, son útiles las siguientes definiciones. Consideramos, el vector  $\mathbf{1} = [1 \dots 1]^\top \in \mathbb{R}^m$  de modo que  $\mathbf{1}\mathbf{1}^\top \in \mathbb{R}^{m \times m}$  es una matriz cuadrada de unos.

**Definición 3.** Definimos la matriz de centrado como

$$H = I - \frac{1}{m} \mathbf{1}\mathbf{1}^\top$$

Recordemos que los datos de entrada, según (1), están agrupados en una matriz  $X \in \mathbb{R}^{d \times p}$ . Entonces, para  $m$  de tamaño apropiado, el producto  $HX$  resta a cada elemento de  $X$  el promedio de su renglón. Mientras que  $XH$  resta a cada elemento de  $X$  el promedio de su columna. Por lo tanto, tenemos las siguientes definiciones.

**Definición 4.** Dada una matriz  $A \in \mathbb{R}^{d \times p}$ .

Definimos a la matriz de centrado *derecho* como el producto  $AH$ . Dicho producto puede ser escrito como

$$AH = A - \mu_{cols}$$

donde

$$\mu_{cols} = \frac{1}{d} \sum_{i=1}^d r_i$$

y  $r_i$  denota el renglón  $i = 1, \dots, d$  de  $A$ .

Definimos a la matriz de centrado *izquierdo* como el producto  $HA$ . El cual, a su vez, puede ser escrito como

$$HA = (A^\top - \mu_{rows})^\top$$

donde

$$\mu_{rows} = \frac{1}{p} \sum_{i=1}^p c_i$$

y  $c_i$  denota la columna  $i = 1, \dots, p$  de  $A$ .

*Nota 1.* Las definiciones anteriores implican que la matriz de centrado es única solamente en el caso de que la matriz  $A$  sea una matriz cuadrada. Sin embargo, la definición (3), funciona para definir tanto a la matriz de centrado izquierdo, como a la matriz de centrado derecho, para un valor de  $m$  apropiado.

Con esto, podemos dar definiciones análogas a (3.6), (3.7) y (3.8), para múltiples datos.

$$\check{X} = XH = X - \mu_{cols} \quad (3.9)$$

$$\tilde{X} = U^\top \check{X} \quad (3.10)$$

$$\hat{X} = UU^\top \check{X} + \mu_{cols} = U\tilde{X} + \mu_{cols} \quad (3.11)$$

Con interpretaciones análogas a sus versiones para un solo dato.

### 3.1.3. La matriz de Covarianza y Descomposición en Vectores Propios

La matriz de Covarianza surge naturalmente al proyectar un dato en una sola dirección. Para ello consideramos la proyección de  $x \in \mathbb{R}^d$  en una sola dirección  $u \in \mathbb{R}^d$ , es decir,  $U = [u] \in \mathbb{R}^{d \times 1}$ .

Proyectando en el subespacio, sin agregar la media, tenemos

$$\hat{x} = uu^\top \check{x}$$

Con la norma  $\ell_2$  al cuadrado

$$\begin{aligned}\|\hat{x}\|_2^2 &= \|uu^\top \check{x}\|_2^2 \\ &= (uu^\top \check{x})^\top (uu^\top \check{x}) \\ &= \check{x}uu^\top uu^\top \check{x} \\ &= \check{x}uu^\top \check{x}\end{aligned}$$

donde  $u^\top u = 1$ , pues  $u$  es unitario

$$\begin{aligned}&= (\check{x}^\top u)(u^\top \check{x}) \\ &= (u^\top \check{x})(\check{x}^\top u)\end{aligned}$$

pues ambos términos son de tamaño  $1 \times 1$

$$= u^\top \check{x} \check{x}^\top u$$

Por lo tanto

$$\|\hat{x}\|_2^2 = u^\top \check{x} \check{x}^\top u \quad (3.12)$$

Ahora, consideramos  $n$  datos  $X$  y su correspondiente proyección  $\hat{X}$  en una sola dirección  $u$ . Entonces

$$\begin{aligned}\sum_{i=1}^n \|\hat{x}_i\|_2^2 &= \sum_{i=1}^n u^\top \check{x}_i \check{x}_i^\top u \\ &= u^\top \left( \sum_{i=1}^n \check{x}_i \check{x}_i^\top \right) u\end{aligned}$$

Es fácil verificar que  $H$  es una matriz simétrica e idempotente. Teniendo esto en cuenta, note que la suma en la expresión es equivalente a:

$$\begin{aligned}\sum_{i=1}^n \check{x}_i \check{x}_i^\top &= \check{X} \check{X}^\top \\ &= (XH)(H^\top X^\top) \\ &= XHHX^\top \\ &= XHX^\top\end{aligned}$$

Que es justamente la matriz de covarianza.

**Definición 5.** Definimos la matriz de covarianza como:

$$S = XHX^\top$$

Si los datos ya están centrados, se reduce a:

$$S = XX^\top$$

Además,  $S \in \mathbb{R}^{d \times d}$ .

Finalmente, consideramos el caso de la proyección de múltiples datos  $X \in \mathbb{R}^{d \times n}$  en múltiples direcciones  $U \in \mathbb{R}^{d \times p}$ ,  $p > 1$ . Si  $\check{X}$  es la proyección de  $X$  en  $\text{col}(U)$ , sin agregar la media, entonces

$$\hat{X} = UU^\top \check{X}$$

En este caso, usamos el cuadrado de la norma de Frobenius:

$$\begin{aligned} \|\hat{X}\|_F^2 &= \|UU^\top \check{X}\|_F^2 \\ &= \text{tr}((UU^\top \check{X})^\top (UU^\top \check{X})) \\ &= \text{tr}(\check{X}^\top UU^\top UU^\top \check{X}) \\ &= \text{tr}(\check{X}^\top U(U^\top U)U^\top \check{X}) \end{aligned}$$

Como  $U$  es ortogonal  $U^\top U = I$

$$\begin{aligned} &= \text{tr}(\check{X}^\top UU^\top \check{X}) \\ &= \text{tr}(U^\top \check{X} \check{X}^\top U) \end{aligned}$$

Por (5), tenemos que

$$\|\hat{X}\|_F^2 = \text{tr}(U^\top S U) \tag{3.13}$$

El siguiente problema de optimización se reduce a una descomposición en valores propios de la matriz de covarianza,  $S$ . A la vez que encuentra las direcciones principales  $\{u_i\}_{i=1}^p$  que maximizan  $\|\hat{X}\|_F^2$ . Es decir, la matriz  $U$  maximiza la longitud de la reconstrucción  $\|\hat{X}\|_F^2$  y por (3.13) maximiza la varianza de  $X$ .

$$\begin{aligned} &\text{máx}_U \quad \text{tr}(U^\top S U) \\ &\text{sujeto a} \quad U^\top U = I \end{aligned}$$

Donde la restricción asegura que  $U$  es ortogonal.

El cual se reduce a la descomposición en valores propios de  $S$ :

$$SU = U\Lambda \quad (3.14)$$

Donde  $\Lambda \in \mathbb{R}^{p \times p}$  es una matriz diagonal de los valores propios de  $S$ , en orden decreciente, y  $U$  la matriz de vectores propios. Llámamos a las columnas de la proyección  $\check{X}$ , componentes principales.

En este punto, no necesariamente hemos reducido la dimensión. En PCA, la reducción de dimensión no se realiza con la descomposición en valores propios de  $S$ , sino con la cantidad columnas de componentes principales que decidimos conservar.

#### 3.1.4. El rango de la matriz de covarianza y los componentes a retener

Hay dos casos para  $\check{X} \in \mathbb{R}^{d \times p}$ :

1. Si  $d \geq n$ , es decir, la dimensión de los datos originales es mayor que el número de puntos. En este caso  $\text{rank}(S) = \text{rank}(\check{X}\check{X}^\top) \leq n - 1$ .
2. Si  $d \leq n - 1$ , entonces  $\text{rank}(\check{X}) = \text{rank}(\check{X}^\top)$  y por lo tanto  $\text{rank}(S) \leq d$ .

Los anteriores casos imponen restricciones al rango de la matriz de covarianza  $S$ . Como se expone a continuación, el rango de  $S$  tiene consecuencias directas sobre  $U$ .

1. Si  $\text{rank}(S) = p$ , entonces  $U \in \mathbb{R}^{d \times d}$ . Es decir, la dimensión del subespacio es la misma que la del espacio original. En este caso, como  $U$  es una matriz cuadrada ortogonal tiene determinante  $+1$  o  $-1$ , en el primer caso se trata de una rotación, mientras que en el segundo de una reflexión.
2. Si  $\text{rank}(S) < d$  y  $n > d$ , decimos que hay suficientes puntos, pero yacen en un subespacio y no llenan el espacio original en cada dirección. Así,  $U \in \mathbb{R}^{d \times p}$  y  $\text{rank}(U) = p < d$ , por lo tanto,  $\text{rank}(UU^\top) = p < d$ .
3. Si  $\text{rank}(S) \leq n - 1 < d$ , no hay suficientes puntos para representar el espacio original. De forma idéntica al caso anterior,  $U \in \mathbb{R}^{d \times p}$  y  $\text{rank}(U) = p < d$ , por lo tanto,  $\text{rank}(UU^\top) = p < d$ .

En cualquier caso, puede ser necesario truncar  $U \in \mathbb{R}^{d \times q}$ ,  $q < p$ . Cuando truncamos  $U$ , buscamos preservar los vectores propios más relevantes. Este puede ser el caso, cuando la varianza es significativamente más grande en  $q < \text{rank}(S)$  direcciones, por lo que decidimos ignorar las direcciones restantes. Siempre que  $U \in \mathbb{R}^{d \times q}$  sea ortonormal, es posible hallar  $U^\top \in \mathbb{R}^{q \times p}$  tal que  $U^\top U = I$ .

Para decidir si debemos truncar  $U$ , existen métodos gráficos como *scree plot* que grafica los valores propios contra los componentes principales ordenados

por valores propios. Otra forma es usar la proporción:

$$\frac{\lambda_j}{\sum_{k=1}^d \lambda_k}$$

En cualquier caso, debemos definir un valor a partir del cual empezamos a descartar vectores principales.

## 4. Kernel PCA

En 1904, David Hilbert propuso su trabajo acerca de kernels, más tarde, Erhard Schmidt's propuso ecuaciones integrales del tipo Fredholm. Está fue la introducción del espacio de Hilbert. Posteriormente, James Mercer trabajó también en espacios de Hilbert y enunció su teorema en 1909 (Ghojogh y col., 2021). En Machine Learning, los espacios de Hilbert con kernel reproducible (RKHS) resultaron ser de gran utilidad pues, un kernel, funciona como medida de similitud en un espacio donde arrastramos los datos, esperando que sea más fácil hallar una proyección apropiada.

En esta sección se desarrolla la teoría esencial sobre espacios de Hilbert con el fin de generalizar el algoritmo de análisis de componentes principales (PCA), a uno capaz de lidiar con conjuntos de datos donde una proyección lineal no es la mejor para hallar una representación en dimensión baja.

### 4.1. Espacios de Hilbert con Kernel Reproducible (RKHS)

Como indica el nombre, un espacio de Hilbert con kernel reproducible (RKHS a partir de ahora) tiene tres características: Es un espacio Hilbert, contiene Kernels y los kernels cumplen la propiedad reproducible. Por lo tanto, estas tres propiedades serán nuestro punto de partida.

**Definición 6.** Un conjunto  $\mathcal{H}$  es un espacio de Hilbert si cumple las siguientes propiedades:

- I Es un espacio lineal.
- II Es un espacio con producto interno, denotado  $\langle \cdot, \cdot \rangle$ .
- III Es un espacio métrico completo, con la métrica  $\rho(f, g) = \|f - g\|$ , donde  $\|f\| = \sqrt{\langle f, f \rangle}$ .
- IV Es un espacio separable.

Son necesarios algunos comentarios sobre la definición anterior. En primera, las condiciones (I) y (II) son equivalentes a decir que  $\mathcal{H}$  es un espacio euclideo. Luego, la condición (IV) a menudo se omite, de forma que  $\mathcal{H}$  puede no ser separable; pero, si es separable, significa que contiene un subconjunto denso numerable.

Por lo anterior podemos dar la siguiente definición de espacio de Hilbert.

**Definición 7.** Un espacio de Hilbert  $\mathcal{H}$  es un espacio con producto interno que es un espacio métrico completo con respecto a la norma inducida por el producto interno.

Una proposición importante sobre los espacios de Hilbert separables de dimensión infinita es que todos ellos son isomorfos, por esta razón a menudo se identifica al espacio  $l_2$  como el espacio de Hilbert. Si embargo, es importante notar

que esto no es cierto en el caso de los RKHS donde cada kernel genera un RKHS distinto.

Siguiendo el programa que hemos trazado, damos la definición de kernel (de Mercer).

**Definición 8.** Una función  $k : \mathcal{X}^2 \rightarrow \mathbb{R}$  es un kernel (de Mercer) o función kernel si:

- I Es simétrica, es decir,  $k(x, y) = k(y, x)$ .
- II Su matriz kernel correspondiente  $\mathbf{K}(i, j) := k(x_i, x_j) \forall i, j \in \{1, \dots, n\}$  es positiva semi-definida:  $\mathbf{K} \succeq 0$ .

La matriz kernel en (II), es una matriz cuadrada  $\mathbf{K} \in \mathbb{R}^{n \times n}$  también llamada matriz de Gram o matriz Gramiana.

Antes de enunciar la propiedad reproducible, es conveniente dar la definición de RKHS.

**Definición 9.** Un espacio de Hilbert con kernel reproducible es un espacio de Hilbert  $\mathcal{H}$  que consta de funciones  $f : \mathcal{X} \rightarrow \mathbb{R}$  con kernel reproducible  $k : \mathcal{X}^2 \rightarrow \mathbb{R}$  donde  $k(x, \cdot) \in \mathcal{H}$  y  $f(x) = \langle k(x, \cdot), f \rangle$ .

Una definición equivalente a la anterior de RKHS es la siguiente.

**Definición 10.** Sea  $\mathcal{H}$  un espacio de Hilbert que consiste de funciones reales en un subconjunto  $\mathcal{X} \subset \mathbb{R}^d$  y considere el funcional evaluación puntual  $E_x : \mathcal{H} \rightarrow \mathbb{R}$  dado por

$$E_x(f) = f(x), \quad f \in \mathcal{H}$$

Decimos que  $\mathcal{H}$  es un RKHS si  $E_x$  es acotado —por lo tanto continuo— para cada  $x \in \mathcal{X}$ . Esto es

$$|E_x(f)| \leq c \|f\|_{\mathcal{H}}, \quad \forall f \in \mathcal{H}$$

La propiedad al final de la definición (9) es llamada *propiedad reproducible*. La razón de esto es que reproduce el funcional evaluación  $E_x$  de la definición (10), es decir:

Para  $y \in \mathcal{X}$  y  $f \in \mathcal{H}$  se tiene

$$f(y) = E_y(f) = \langle f, k(y, \cdot) \rangle = \langle f, k(\cdot, y) \rangle \quad (4.1)$$

Sin embargo, esta propiedad es más general como se verá más adelante en (2). Para ello, es útil el siguiente resultado que involucra la base de los RKHS. En específico,

$$\begin{aligned} \mathcal{H} &= \left\{ f(\cdot) = \sum_{i=1}^n \alpha_i k(x_i, \cdot) \right\} \\ &= \left\{ f(\cdot) = \sum_{i=1}^n \alpha_i k_{x_i}(\cdot) \right\} \end{aligned} \quad (4.2)$$



donde  $\alpha_i$  son escalares.

Este resultado muestra que un RKHS tiene una base de kernels. Aunque estos son vistos como funciones de una sola variable  $k_{x_i}(\cdot) = k(x_i, \cdot)$ .

El siguiente resultado es de utilidad para calcular el producto interno de dos miembros en  $\mathcal{H}$ .

**Proposición 1.** *Si  $\mathcal{H}$  es un RKHS y  $f, g \in \mathcal{H}$  entonces*

$$\langle f, g \rangle = \sum_{j=1}^n \sum_{i=1}^n \alpha_i \beta_j k(x_i, y_j)$$

**Demostración** Por (4.2), podemos escribir

$$\begin{aligned} f(x) &= \sum_{i=1}^n \alpha_i k(x_i, x) \\ g(y) &= \sum_{j=1}^n \beta_j k(y_j, y) \end{aligned}$$

Reemplazando en el producto interno obtenemos

$$\langle f, g \rangle = \left\langle \sum_{i=1}^n \alpha_i k(x_i, x), \sum_{j=1}^n \beta_j k(y_j, y) \right\rangle$$

por simetría de  $k$

$$\begin{aligned} &= \left\langle \sum_{i=1}^n \alpha_i k(x_i, x), \sum_{j=1}^n \beta_j k(y, y_j) \right\rangle \\ &= \sum_{i=1}^n \sum_{j=1}^n \alpha_i \beta_j \langle k(x_i, x), k(y, y_j) \rangle \\ &= \sum_{i=1}^n \sum_{j=1}^n \alpha_i \beta_j \langle k_{x_i}(\cdot), k_{y_j}(\cdot) \rangle \end{aligned}$$

Note que  $h(z) = k_{y_j}(z) \in \mathcal{H}$  y por lo tanto podemos aplicar la propiedad al final de la definición (9), es decir,  $h(z) = \langle k(z, \cdot), h \rangle$

$$\begin{aligned} &= \sum_{i=1}^n \sum_{j=1}^n \alpha_i \beta_j h(x_i) \\ &= \sum_{i=1}^n \sum_{j=1}^n \alpha_i \beta_j k(x_i, y_j) \end{aligned}$$

Con esto, podemos dar la propiedad reproducible.

**Proposición 2.** Si  $\mathcal{H}$  es un RKHS, entonces para cada  $f \in \mathcal{H}$  existe un kernel  $g \in \mathcal{H}$  tal que

$$\langle f(x), g(x) \rangle = f(x)$$

**Demostración** Por (4.2), podemos escribir

$$f(x) = \sum_{i=1}^n \alpha_i k(x_i, x)$$

Consideramos el kernel  $g(x) = k(x, x)$  ó  $g(x) = k_x(x)$ . Entonces

$$\begin{aligned} \langle f, g \rangle &= \left\langle \sum_{i=1}^n \alpha_i k(x_i, x), g(x) \right\rangle \\ &= \left\langle \sum_{i=1}^n \alpha_i k(x_i, x), k(x, x) \right\rangle \end{aligned}$$

por la proposición anterior

$$\begin{aligned} &= \sum_{i=1}^n \alpha_i k(x_i, x) \\ &= f(x) \end{aligned}$$

Este resultado es especialmente útil, pues en las aplicaciones —a menudo— no conocemos la posición de los puntos en  $\mathcal{H}$  pero sí el kernel. Por lo tanto, la matriz de Mercer  $\mathbf{K}$  es una matriz que nos dice la similitud (o disimilitud) de los puntos en  $\mathcal{H}$ .

Finalizamos esta sección con una discusión sobre el término *kernelización*. Consideremos un algoritmo lineal aplicado sobre un conjunto de datos no-lineal. Es evidente que el algoritmo no se desempeñará correctamente sobre el conjunto de datos. Para corregir esto, según (Ghojogh y col., 2023), tenemos dos opciones: Modificar el algoritmo para que sea capaz de manejar datos no-lineales o modificar el conjunto de datos para que presente un patrón más lineal. La última es justamente la idea de la que surge el concepto de *kernelización*. En ciencia de datos, *kernelización* se refiere a usar los datos “arrastrados” a un RKHS como entrada de un algoritmo, en lugar de usar los datos “crudos”.

Existen dos métodos de *kernelización*: el truco del kernel (*kernel trick*) y *kernelización* usando teoría de la representación. Nosotros nos enfocaremos en el primero porque es justamente el enfoque detrás de kernel PCA. El *kernel trick* intenta reformular el algoritmo de modo que no aparezcan instancias de los datos en crudo  $x$ , dejando el algoritmo en términos del producto interno:  $x^\top x$ ,  $x^\top X$ ,  $X^\top x$  ó  $X^\top X$  y reemplazar dichas instancias por un kernel. Pues, como mencionamos arriba, es fácil calcular el kernel usando únicamente el producto interno.

## 4.2. Kernel PCA

Muchos métodos de reducción de dimensión se pueden reducir a kernel PCA, usando los vectores propios (en general, funciones propias) para arrastrar los datos al espacio de características.

*Ejemplo 3.* Kernel PCA es equivalente a PCA —por el *kernel trick*— si usamos el kernel

$$K = X^\top X$$

Dicha equivalencia se vuelve más aparente si en el algoritmo original de PCA descomponemos la matriz de covarianza en valores singulares en lugar de descomponerla en valores propios.

*Ejemplo 4.* El algoritmo de escalado multidimensional (MDS) es equivalente a kernel PCA, haciendo:

$$K = -\frac{1}{2}HDH$$

Donde  $D$  es la matriz de distancia euclídeana al cuadrado y  $H$  es la matriz de centrado.

Si en lugar de usar la distancia euclídeana usamos la distancia geodésica, kernel PCA es equivalente a isomap con el kernel:

$$K = -\frac{1}{2}HD^{(g)}H$$

Donde  $D^{(g)}$  es la matriz de distancias geodésicas.

Sin embargo, el uso de kernels no se limita a métodos de reducción de dimensión. También puede usarse en el análisis de conglomerados (*clustering*).

*Ejemplo 5.* El algoritmo de *spectral clustering* usa el kernel

$$K_{ij} = \frac{W_{ij}}{\sqrt{D_{ii}D_{jj}}}$$

Donde  $W \in \mathbb{R}^{n \times n}$  es la matriz de adyacencia.

La principal ventaja de PCA no es computacional sino conceptual. Actúa como un marco unificador de una gran cantidad de métodos espectrales de reducción de dimensión y tiene aplicaciones en otras áreas de la ciencia de datos.

### 4.2.1. Preliminares

De manera informal, la idea de kernel PCA es arrastrar los datos que —asumimos— yacen en una variedad no-lineal hacia un espacio donde —con suerte— sigan una distribución lineal o aproximadamente lineal; y, una vez ahí, aplicar PCA. Para aclarar estos términos damos la siguientes definiciones.

**Definición 11.** Llamamos a  $\mathcal{X}$  espacio original (*input space*). A menudo  $\mathcal{X} = \mathbb{R}^d$ .

Llamamos al espacio de Hilbert  $\mathcal{H}$  espacio de las características (*feature space*). Puede tratarse de un espacio de dimensión finita o infinita, decimos que  $\mathcal{H}$  es  $t$ -dimensional.

El mapeo  $\phi : \mathcal{X} \rightarrow \mathcal{H}$  tal que  $x \mapsto \phi(x)$  es llamado función de arrastre (*pulling function* o *feature map*).

Es importante diferenciar entre el espacio original  $\mathcal{X}$  y el conjunto de datos en forma de matriz  $X = [x_1 \dots x_n]$ . Teniendo esto en mente, damos la siguiente definición.

**Definición 12.** La aplicación de la función de arrastre  $\phi$  sobre el conjunto de datos  $X$  está definida como

$$\Phi(X) = [\phi(x_1) \dots \phi(x_n)] \quad (4.3)$$

Siendo  $t \times n$  dimensional.

Usando el teorema de Mercer se puede mostrar que

$$k(x, y) = \langle \phi(x), \phi(y) \rangle = \phi(x)^\top \phi(y) \quad (4.4)$$

Este resultado ilustra una ventaja de la kernelización: para calcular el kernel solo hace falta el producto interno; si recordamos que el producto interno es una medida de similitud en el espacio de las características, entonces no es necesario conocer la función de arrastre (por lo tanto la posición exacta de los datos en el espacio de características) para tener una idea de la similitud entre los datos.

Analogamente para el conjunto de datos

$$\mathbf{K} = \langle \Phi(X), \Phi(X) \rangle = \Phi(X)^\top \Phi(X) \quad (4.5)$$

Este resultado es fácil de ver, razonando como sigue:

$$\begin{aligned} \Phi(X)^\top \Phi(X) &= [\phi(x_1) \dots \phi(x_n)]^\top [\phi(x_1) \dots \phi(x_n)] \\ &= [\phi(x_1)^\top \dots \phi(x_n)^\top] [\phi(x_1) \dots \phi(x_n)] \\ &= [\phi(x_1)^\top \phi(x_1) \dots \phi(x_n)^\top \phi(x_n)] \\ &= [k(x_i, x_j)] \\ &= \mathbf{K} \end{aligned}$$

Finalmente, al igual que en PCA, un paso previo es centrar los datos. En el caso de kernel PCA, no centramos los datos en crudo sino los datos una vez arrastrados. A este proceso se le llama centrado del kernel.

**Definición 13.** Definimos los datos centrados en el espacio de características como

$$\check{\Phi}(X) = \Phi(X)H$$

Donde  $H$  es la matriz de centrado.

Centrar los datos en el espacio de la característica equivale al doble centrado de  $K$ .

**Proposición 3.**

$$\check{K} = HKH$$

**Demostración**

$$\begin{aligned}\check{K} &= \check{\Phi}^\top(X)\check{\Phi}(X) \\ &= (\Phi(X)H)^\top(\Phi(X)H) \\ &= (H^\top\Phi^\top(X))\Phi(X)H \\ &= H^\top\Phi^\top(X)\Phi(X)H \\ &= H^\top KH \\ &= HKH\end{aligned}$$

El kernel de (3) es llamado kernel doble-centrado y se trata de los datos centrados en el espacio de características.

#### 4.2.2. Proyección en el espacio de características

El paso de la proyección en el espacio de características es sencillo si estamos familiarizados con el problema de los vectores propios, vectores propios generalizados y la descomposición en valores singulares. Por esa razón empezamos presentando dichos problemas y algunos resultados de utilidad.

**Definición 14.** El problema de valores propios de una matriz simétrica  $A \in \mathbb{R}^{d \times d}$  consiste en hallar  $\Lambda = \text{diag}[\lambda_1, \dots, \lambda_d]$  diagonal y  $\Phi = [\phi_1, \dots, \phi_d]$  una matriz de vectores columna —ambas de tamaño  $d \times d$ — tales que

$$A\Phi = \Phi\Lambda$$

Si  $A$  es simétrica, entonces  $\Phi$  es ortogonal y la condición anterior se convierte en

$$A = \Phi\Lambda\Phi^{-1} = \Phi\Lambda\Phi^\top \quad (4.6)$$

A la ecuación anterior se le conoce como descomposición en valores propios de  $A$  o descomposición espectral de  $A$ .

Podemos generalizar el problema de descomposición espectral como sigue.

**Definición 15.** Considere dos matrices simétricas  $A, B \in \mathbb{R}^{d \times d}$ , el problema de valores propios del par  $(A, B)$  es hallar  $\Phi, \Lambda$  como en (14) tales que

$$A\Phi = B\Phi\Lambda \quad (4.7)$$

Otra manera de descomponer una matriz es en valores singulares (SVD) la cual tiene dos formas: descomposición completa e incompleta. No es el propósito el dar un método para descomponer la matriz, daremos por hecho que la descomposición es posible y enunciaremos algunas propiedades de utilidad.

**Definición 16.** Considere una matriz  $A \in \mathbb{R}^{\alpha \times \beta}$ , SVD completo descompone la matriz como:

$$A = U\Sigma V^T \quad (4.8)$$

Donde  $U \in \mathbb{R}^{\alpha \times \alpha}$ ,  $V \in \mathbb{R}^{\beta \times \beta}$  y  $\Sigma \in \mathbb{R}^{\alpha \times \beta}$ . La matriz  $U$  es llamada matriz de vectores singulares izquierdos; la matriz  $V$ , matriz de vectores singulares derechos; mientras que  $\Sigma$  es una matriz diagonal, no necesariamente cuadrada, que contiene los valores singulares en la diagonal principal. Esto implica que el número de valores singulares es  $k = \min\{\alpha, \beta\}$ .

Por otro lado, tenemos SVD incompleto.

**Definición 17.** Considere una matriz  $A \in \mathbb{R}^{\alpha \times \beta}$ , SVD incompleto descompone la matriz como:

$$A = U\Sigma V^T$$

Donde  $U \in \mathbb{R}^{\alpha \times k}$ ,  $V \in \mathbb{R}^{k \times \beta}$  y  $\Sigma \in \mathbb{R}^{k \times k}$  y  $k = \min\{\alpha, \beta\}$ .

En ambos casos,  $U$  y  $V$  son ortogonales y por lo tanto se cumple que

$$\begin{aligned} U^T U &= I \\ V^T V &= I \end{aligned}$$

Pero solamente en el caso de SVD completo se cumple

$$\begin{aligned} U U^T &= I \\ V V^T &= I \end{aligned}$$

Finalmente, presentamos el siguiente resultado que establece una conexión entre la descomposición espectral y la descomposición en valores singulares (SVD).

**Proposición 4.** En ambas formas de descomposición en valores singulares (SVD) de  $A$  se cumplen los siguientes enunciados:

- I Las matrices de vectores singulares derechos/izquierdos son los vectores propios de la descomposición espectral de  $AA^T$  y de  $A^T A$ , respectivamente.
- II Los valores singulares son la raíz cuadrada de los valores propios de  $AA^T$  ó  $A^T A$ .

**Demostración** Sea  $A = U\Sigma V^T$  la descomposición en valores singulares (com-

pleta o incompleta) de  $A$ . Entonces,

$$\begin{aligned} AA^\top &= (U\Sigma V^\top)(U\Sigma V^\top)^\top \\ &= U\Sigma V^\top V\Sigma U^\top \\ &= U\Sigma^2 U^\top \end{aligned}$$

Note que  $\Sigma^2$  es diagonal porque  $\Sigma$  es diagonal y por lo tanto, la ecuación anterior es la descomposición espectral de  $AA^\top$ , donde  $U$  es la matriz de vectores propios y  $\Sigma^2$  la una matriz diagonal que contiene los valores propios. Esto último implica que los valores propios de  $AA^\top$  son el cuadrado de los valores singulares de  $A$ .

La demostración para  $A^\top A$  es similar:

$$\begin{aligned} A^\top A &= (U\Sigma V^\top)^\top (U\Sigma V^\top) \\ &= V\Sigma U^\top U\Sigma V^\top \\ &= V\Sigma^2 V^\top \end{aligned}$$

Con interpretación análoga a la de  $AA^\top$ .

Los resultados anteriores son suficientes para presentar la proyección en el espacio de características usando kernel PCA. La idea es hacer SVD incompleto sobre  $\check{\Phi}(X)$ . Es importante recordar que, a partir de ahora,  $\phi$  es la función de arrastre,  $X$  es el conjunto de datos,  $\mathbf{K}_x$  es la matriz kernel de dicho conjunto y  $H$  la matriz de centrado. Todas ellas según fueron definidas en la sección de preeliminarios (4.2.1).

Para el conjunto de datos  $X$ , tenemos la matriz doble-centrada:

$$\check{\mathbf{K}}_x = H\mathbf{K}_xH = \check{\Phi}(X)^\top \check{\Phi}(X) \quad (4.9)$$

Como ya dijimos, la idea usar SVD incompleto sobre  $\check{\Phi}(X)$ , pero esta idea no nos llevará muy lejos si no conocemos  $\Phi$ , que es el caso usual. Sin embargo, el kernel sí es conocido y por lo tanto aplicamos descomposición espectral a él.

$$\check{\mathbf{K}}_x V = V\Lambda \quad (4.10)$$

Note que  $\check{\mathbf{K}}_x = \check{\Phi}(X)^\top \check{\Phi}(X)$ , entonces por la proposición (4), los vectores propios  $V$  son justamente los vectores singulares derechos de la SVD incompleta (pues  $\Lambda$  es cuadrada) de  $\check{\Phi}(X)$  y  $\Lambda$  consta del cuadrado de los valores singulares de dicha descomposición. Con este enfoque, es imposible conocer los vectores singulares izquierdos de la descomposición de  $\check{\Phi}(X)$  sin conocer  $\Phi$ . Sin embargo, veremos que no es necesario conocerlos.

Dada el SVD incompleto de  $\breve{\Phi}(X)$  como sigue

$$\breve{\Phi}(X) = V\Sigma U \quad (4.11)$$

Tenemos que

$$\breve{\mathbf{K}}_x V = V\Sigma^2 \quad (4.12)$$

Con esto, la proyección de  $X$  en kernel PCA está dada por:

$$\begin{aligned} U^\top \breve{\Phi}(X) &= U^\top U \Sigma V^\top \\ &= \Sigma V^\top \end{aligned}$$

Es decir

$$\tilde{\Phi}(X) = U^\top \breve{\Phi}(X) = \Sigma V^\top \quad (4.13)$$



## 5. Ejemplo

### 5.1. Preliminares

Antes de proceder con un ejemplo, es importante hacer algunos comentarios sobre los métodos basados en kernels y explicar conceptos que no se desarrollarán a fondo pero que usaremos en los ejemplos.

#### 5.1.1. Kernels usuales

En esta sección damos la definición y propiedades de algunos kernels de utilidad, tanto para kernel PCA como para otros métodos basados en kernels.

**Definición 18.** El kernel lineal está definido como

$$k(x, y) = x^\top y$$

Este kernel tiene la propiedad de que la función de arrastre es conocida, pues  $k(x, y) = \phi(x)^\top \phi(y) = x^\top y$  y por lo tanto  $\phi(x) = x$ . Es decir, con este kernel los datos permanecen en el espacio original.

**Definición 19.** Definimos el kernel polinomial como  $k(x, y) = (\gamma x^\top y + c)^d$

**Definición 20.** Para  $\sigma > 0$ , el kernel Gaussiano se define como

$$k(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

El kernel Gaussiano suele funcionar bien con datos de distribución circular, además, valores chicos de  $\sigma$  hacen que la matriz kernel tienda a la identidad, mientras que valores grandes hacen que el kernel tienda a ser constante. Los datos en el *feature space* tienen norma 1 pues  $k(x, x) = \exp(0) = 1$ .

**Definición 21.** El kernel coseno está definido como

$$k(x, y) = \cos(x, y) = \frac{x^\top y}{\|x\| \|y\|}$$

Este kernel mide la similaridad de ángulos, proyecta los datos en una hiper-ésfera unitaria.

**Definición 22.** Si  $d : \mathcal{X}^2 \rightarrow \mathbb{R}$  es una métrica y  $D = (d(x_i, x_j))$  es la matriz de la métrica aplicada a cada par de puntos del conjunto de datos, entonces  $K = -\frac{1}{2}HDH$  es un kernel.

Esta definición no dice como construir un kernel a partir de una métrica, será de utilidad para establecer una equivalencia entre el algoritmo no-lineal Isomap y kernel PCA.

### 5.1.2. Error de Reconstrucción y *Trustworthiness*

En el caso de PCA usual, sabíamos que se trataba de la mejor proyección lineal de los datos. Pues es equivalente al problema de optimización:

$$\begin{aligned} \min_U \|\check{X} - UU^\top \check{X}\|^2 \\ \text{sujeto a } U^\top U = I \end{aligned}$$

Es decir, PCA encuentra la proyección lineal que minimiza el error de reconstrucción.

En el ejemplo, evaluaremos varias proyecciones con distintos kernels que a su vez pueden diferir en los parámetros. No es posible, en general, evaluar las proyecciones en base al error de reconstrucción porque —con la teoría que hemos desarrollado— no siempre es posible calcular dicho error. Teóricamente la reconstrucción es  $\hat{\Phi}(X) = \check{\Phi}(X)VV^\top + \mu_x$ , por lo que sin conocer  $\Phi$  es imposible hallar el error de reconstrucción.

En su lugar, usamos el índice de *trustworthiness*, definido como:

$$T(k) = 1 - \frac{2}{nk(2n - 3k - 1)} \sum_{i=1}^n \sum_{j \in \mathcal{N}_i^k} \max(0, (r(i, j) - k))$$

Donde  $\mathcal{N}_i^k$  es el conjunto de los  $k$  vecinos más cercanos al punto  $x_i$  en la proyección y  $r(i, j)$  es el  $r(i, j)$ -ésimo vecino más cercano en el espacio original.

Si entrar en detalles, se trata de un índice que mide qué tanto se mantiene la estructura de vecindades antes y después de la proyección. Además, penaliza el hecho de que, en la proyección, aparezcan vecinos que no estaban en las vecindades originales. Pues la aparición de nuevos vecinos puede inducir vecindades de puntos que no son similares entre sí.

## 5.2. Desenvolviendo una curva S

Usaremos, para la generación y manipulación del conjunto de datos, las siguientes librerías de Python:

1. Pandas
2. Numpy
3. SKLearn

La librería `sklearn` contiene la implementación de Kernel PCA, además de herramientas para obtener la matriz kernel y para calcular el índice de *trustworthiness*.

Empezamos generando el conjunto de datos: mil puntos de una curva S, cuya gráfica se muestra en la figura (3). En el código, `X` es el conjunto de datos, mientras que `t` contiene la posición relativa en la curva.

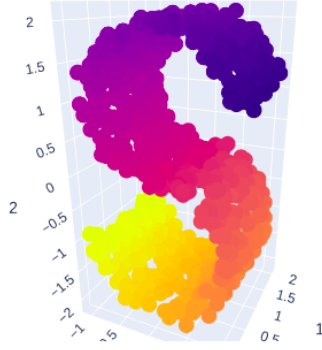


Figura 3: Gráfica de los puntos sobre la curva S

|    | kernel | gamma   | degree    | trustworthiness |          |
|----|--------|---------|-----------|-----------------|----------|
| 1  | 0      | rbf     | 0.372759  | NaN             | 0.898869 |
| 2  | 1      | sigmoid | 10.000000 | NaN             | 0.896587 |
| 3  | 2      | rbf     | 0.848343  | NaN             | 0.896172 |
| 4  | 3      | rbf     | 1.930698  | NaN             | 0.893615 |
| 5  | 4      | sigmoid | 4.393971  | NaN             | 0.883386 |
| 6  | 5      | cosine  | NaN       | NaN             | 0.881061 |
| 7  | 6      | poly    | 10.000000 | 5.0             | 0.875199 |
| 8  | 7      | poly    | 0.372759  | 7.0             | 0.871906 |
| 9  | 8      | poly    | 4.393971  | 5.0             | 0.871842 |
| 10 | 9      | rbf     | 0.163789  | NaN             | 0.870786 |

Cuadro 1: Tabla de los diez mejores kernels y sus parámetros

```
1 X, t = make_s_curve(n_samples= 1000, noise=0.05, random_state=0)
```

Mediante una búsqueda por fuerza bruta, probamos todos los kernels sobre el espacio de parámetros  $\gamma \in [0,0001,10]$  y  $d \in \{3,5,7\}$ . En el cuadro (1), se muestran los diez mejores kernels y sus parámetros.

Ajustamos el mejor modelo a los datos y obtenemos la proyección de la figura (4).

Podemos obtener la matriz Kernel con el siguiente código.

```
1 X_scaled['pos'] = t
2 kpca_matrix = pairwise_kernels(X_scaled.sort_values(by='pos'),
3                               metric=best_name, gamma=best_gamma)
```

El resultado se muestra en la figura (5). Donde un color cercano al amarillo indica mayor similitud entre los puntos, es decir, en ella podemos visualizar la

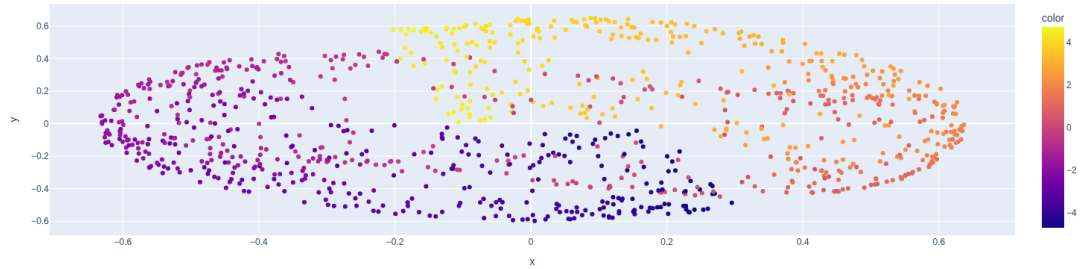


Figura 4: Proyección Usando KPCA con los kernels usuales

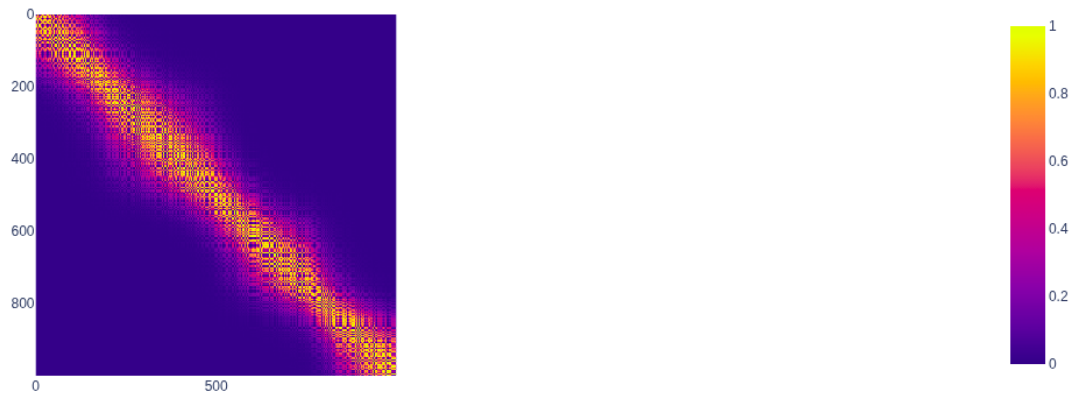


Figura 5: Matriz de Mercer del mejor kernel

estructura de vecindades en la proyección. Notamos que cada punto es similar solamente con sus vecinos más cercano, razón por la cual obtenemos un buen índice de *trustworthiness*, aunque en la proyección es evidente que no se mantiene la estructura global de los datos originales.

Finalmente, evaluaremos el desempeño sobre nuestro conjunto de datos de otro algoritmo no-lineal: Isomap. Se trata de un algoritmo no-lineal de reducción de dimensión que generaliza al algoritmo lineal MDS.

Consiste en los siguientes pasos:

1. Hallar los  $k$  vecinos más cercanos a cada punto en el conjunto de datos.
2. Construir una gráfica uniendo dos puntos si y solo si son vecinos.
3. Hallar la distancia más corta entre cualquiera dos puntos.

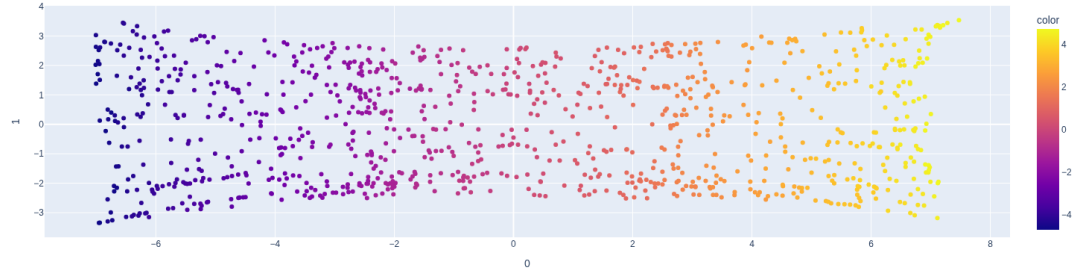


Figura 6: Proyección Obtenida con Isomap

#### 4. Usar MDS para reducir la dimensión.

Con la métrica  $D_{i,j} = \min \sum_{i=1}^l \|r_i - r_{i+1}\|$  y el kernel  $K = -\frac{1}{2}HD^2H$ , se puede mostrar que kernel PCA es equivalente a Isomap.

Este hecho ejemplifica una de las grandes ventajas de kernel PCA: unificar a otros algoritmos de reducción de dimensión dentro de un mismo marco teórico. De esta forma, no es necesario saber los detalles de Isomap para usarlo, solamente el kernel bajo el que Kernel PCA es equivalente.

En *Python*, Isomap se encuentra implementado en la librería **sklearn**. Ajustamos el modelo a los datos con el siguiente código.

```
1 isomap = Isomap(n_neighbors=12, n_components=2, p=1)
2 X_isomap = isomap.fit_transform(X_scaled)
```

Obtenemos un índice de *trustworthiness* de 0.9992801833927661, el mejor hasta ahora. La proyección y la matriz kernel se muestran en la figura (6) y (7), respectivamente.

En base al valor de *trustworthiness* y la gráfica de la proyección, podemos concluir que Isomap preserva la estructura local y global de los datos originales. Efectivamente, “desenvolviendo” la variedad. En la matriz kernel se puede apreciar que la similitud entre puntos es continua entre todos los puntos, razón por la que conserva de mejor forma la estructura global.

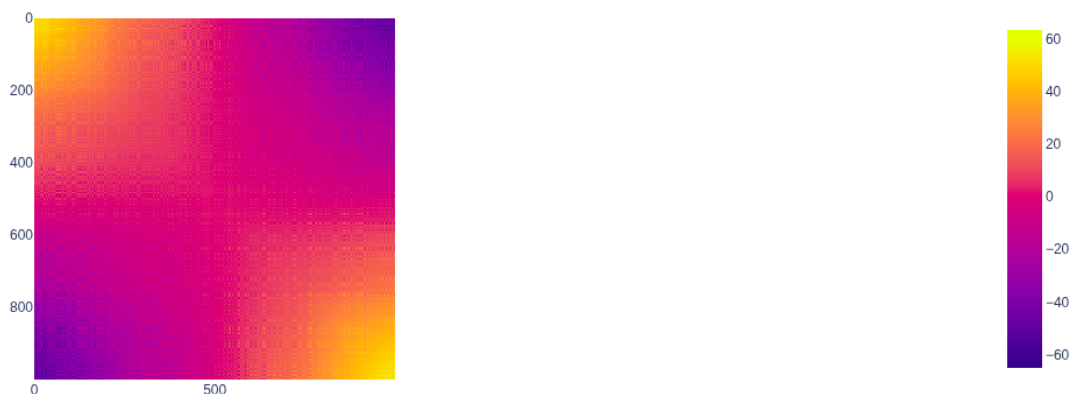


Figura 7: Matriz de Mercer de Isomap

## 6. Conclusiones

A lo largo de este trabajo, hemos explorado el flujo de trabajo del proceso de ciencia de datos —desde la recolección de datos en crudo hasta la toma de decisiones basada en modelos y visualizaciones— enfocándonos en la reducción de dimensión, una etapa crítica tanto para la exploración como para la interpretación de conjuntos de datos con una gran cantidad de variables.

El Análisis de Componentes Principales (PCA) es un algoritmo indispensable por ser el método lineal de reducción de dimensión por excelencia. Comprender su mecánica —basada en encontrar el subespacio que minimiza el error de reconstrucción y maximiza la varianza— establece la base conceptual necesaria para abordar técnicas más complejas. Aunque se trata de un algoritmo relativamente simple, su estudio es importante pues, mediante el uso de funciones kernel y matrices de Mercer, puede construirse una generalización de PCA: kernel PCA.

En este contexto, la teoría de los Espacios de Hilbert con Kernel Reproducible (RKHS) y el Teorema de Mercer fundamentan formalmente a Kernel PCA. Aplicando el *kernel trick*, es posible mapear los datos a un espacio de características de mayor (o infinita) dimensión para aplicar una proyección lineal sin la necesidad de conocer explícitamente la función de arrastre. Como se mostró, Kernel PCA no es solo una generalización no lineal, sino que actúa como un marco unificador bajo el cual otros métodos espectrales (como MDS, Isomap, LLE o incluso el spectral clustering) pueden entenderse como casos particulares bajo selecciones específicas de kernels o matrices kernel.

Finalmente, obtuvimos los siguientes resultados al intentar desdoblar un variedad tridimensional en forma de curva S:

- Los kernels tradicionales en Kernel PCA (como RBF, Gaussiano o Polinomial) demostraron ser eficaces para preservar las relaciones de vecindad local —alcanzando métricas de trustworthiness cercanas al 0.90—, pero fallaron en preservar la geometría y la estructura topológica global de la variedad.
- Por otro lado, la equivalencia de Kernel PCA con Isomap mediante el uso de distancias geodésicas demostró ser superior para este tipo de estructuras no lineales, alcanzando un trustworthiness de 0.9992 y logrando desdoblar con éxito la curva S en dos dimensiones.

En conclusión, los métodos de reducción de dimensión exigen una comprensión de la teoría subyacente para interpretar los resultados y hacer buenas aplicaciones. Si bien las técnicas lineales como PCA siguen siendo herramientas esenciales y eficientes, los métodos no lineales basados en kernels constituyen un marco robusto y unificador para descubrir y preservar la estructura latente en conjuntos de datos complejos.

## Referencias

- Driscoll, T. A., & Braun, R. J. (2022). *Fundamentals of Numerical Computation: Julia Edition*. Society for Industrial; Applied Mathematics. <https://doi.org/10.1137/1.9781611977011>
- Fullstory. (2024). Qualitative vs. quantitative data in research: The difference. <https://www.fullstory.com/blog/qualitative-vs-quantitative-data/>
- Garzon, M., Yang, C.-C., Venugopal, D., Kumar, N., Jana, K., & Deng, L.-Y. (Eds.). (2022). *Dimensionality reduction in data science*. Springer International Publishing.
- Ghojogh, B., Crowley, M., Karray, F., & Ghodsi, A. (2023). *Elements of dimensionality reduction and manifold learning*. Springer.
- Ghojogh, B., Ghodsi, A., Karray, F., & Crowley, M. (2021). Reproducing Kernel Hilbert Space, Mercer's Theorem, Eigenfunctions, Nyström Method, and Use of Kernels in Machine Learning: Tutorial and Survey. <https://arxiv.org/abs/2106.08443>
- IBM. (2025). What is exploratory data analysis? <https://www.ibm.com/think/topics/exploratory-data-analysis>
- Ma, Y., & Fu, Y. (2012). *Manifold learning theory and applications*. Taylor Francis distributor.
- Pearson, K. (1901). LIII. On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11), 559-572. <https://doi.org/10.1080/14786440109462720>
- Rhys, H. (2020). *Machine learning with R, the Tidyverse, and MLR*. Manning Publications.
- Schutt, R., & O'Neil, C. (2014). *Doing data science*. O'Reilly Media.
- Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.
- Venna, J., & Kaski, S. (2001). Neighborhood Preservation in Nonlinear Projection Methods: An Experimental Study. [https://doi.org/10.1007/3-540-44668-0\\_68](https://doi.org/10.1007/3-540-44668-0_68)
- von Luxburg, U. (2007). A Tutorial on Spectral Clustering. <https://arxiv.org/abs/0711.0189>